

Sistema de Gestión de Alumnos

EVIDENCIA TÉCNICA DE DESPLIEGUE Y BITÁCORA DE ERRORES CON CAPTURAS

Esta documentación detalla el proceso completo de desarrollo, despliegue en infraestructura virtual y depuración del sistema distribuido de gestión de alumnos organizado en **3 capas físicas independientes**. Las pruebas y configuraciones se ejecutaron en un entorno controlado con el hipervisor **VirtualBox**, corriendo tres instancias de servidores lógicos basados en **Ubuntu Server 24.04 LTS**.

El esquema respeta estrictamente los principios de diseño multiplataforma:

1. **Capa de Datos (VM1):** Motor relacional MariaDB Server, encargado de resguardar el modelo de datos.
2. **Capa de Negocio (VM2):** Backend RESTful desarrollado sobre Node.js y Express, responsable de la lógica del sistema.
3. **Capa de Presentación (VM3):** Servidor Apache2 encargado de proveer la interfaz de usuario web al cliente.

Diagrama de Red e Infraestructura Virtual

Dado el estricto aislamiento impuesto por las reglas del modo **NAT** individual de VirtualBox, se configuró un direccionamiento dinámico cruzando la interfaz loopback especial del host físico (**10.0.2.2**). Esto posibilitó que los paquetes de datos fluyeran entre las máquinas de forma segura a través de túneles lógicos sin alterar la topología local.



Guía de Instalación y Configuración Paso a Paso

Fase 1: Configuración de la Capa de Datos (VM1)

1. Instalación de MariaDB:

```
sudo apt update && sudo apt install mariadb-server -y
```

2. Direccionamiento de Red Estática:

Archivo editado en `sudo nano /etc/netplan/50-cloud-init.yaml`:

```

network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: false
      addresses:
        - 10.0.2.15/24
      routes:
        - to: default
          via: 10.0.2.2
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]

```

Aplicación de configuraciones: `sudo netplan apply`.

3. Inicialización del Esquema SQL:

```

CREATE DATABASE systema_alumnos;
USE systema_alumnos;

CREATE TABLE alumnos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  apellidos VARCHAR(100) NOT NULL,
  nombre VARCHAR(100) NOT NULL,
  dni VARCHAR(100) UNIQUE NOT NULL
);

CREATE USER 'alejandro'@'%' IDENTIFIED BY '123456';
GRANT ALL PRIVILEGES ON systema_alumnos.* TO 'alejandro'@'%';
FLUSH PRIVILEGES;
EXIT;

```

Fase 2: Configuración del Backend / API Rest (VM2)

1. Depuración y Setup Inicial:

```

sudo apt remove --purge mariadb-server mariadb-client -y && sudo apt autoremove -y
mkdir ~/api-backend && cd ~/api-backend
npm init -y && npm install express mysql2 cors

```

2. **Configuración de Red Estática:** Fije la IP `10.0.2.16/24` a través del archivo YAML de Netplan y aplique los cambios de red correspondientes.

3. **Código de Escucha del Backend (`index.js`):**

```

const express = require('express');
const mysql = require('mysql2/promise');
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

const dbConfig = {
  host: '10.0.2.2',
  user: 'alejandro',
  password: '123456',
  database: 'systema_alumnos',
  port: 3306
};

app.post('/grabaAlumnos', async (req, res) => {
  const { apellidos, nombres, dni } = req.body;
  try {
    const conn = await mysql.createConnection(dbConfig);
    const [rows] = await conn.execute('SELECT count(*) as count FROM alumnos WHERE dni = ?',
[dni]);
    if (rows[0].count > 0) {
      res.json(0);
    } else {
      await conn.execute('INSERT INTO alumnos (apellidos, nombre, dni) VALUES (?, ?, ?)',
[apellidos, nombres, dni]);
      res.json(1);
    }
    await conn.end();
  } catch (error) {
    console.error("Error al grabar:", error.message);
    res.status(500).json(0);
  }
});

app.get('/consultarAlumnos', async (req, res) => {
  try {
    const conn = await mysql.createConnection(dbConfig);
    const [rows] = await conn.execute('SELECT id, apellidos, nombre AS nombres, dni FROM alumnos
ORDER BY apellidos ASC, nombre ASC');
    res.json(rows);
    await conn.end();
  } catch (error) {
    console.error("Error al consultar:", error.message);
    res.status(500).json({ error: 'Error en DB' });
  }
});

app.listen(3454, () => {
  console.log('🚀 Backend Node.js activo en el puerto 3454');
});

```



Bitácora de Errores Comunes, Diagnóstico y Soluciones

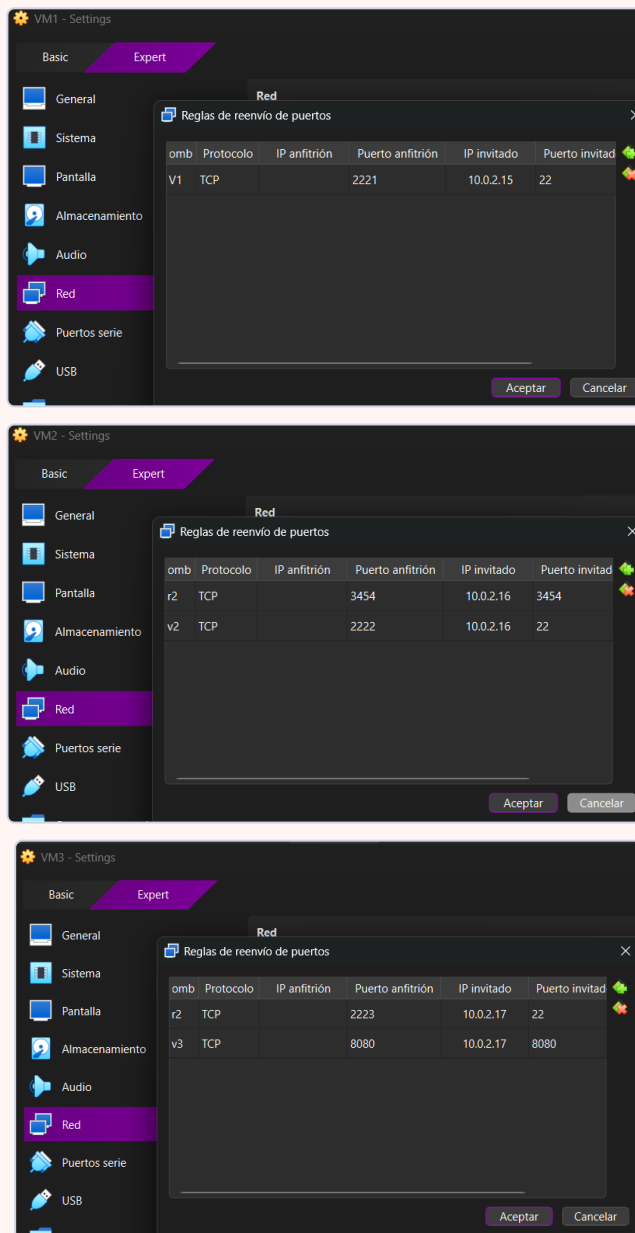
A continuación, se adjunta el compendio analítico de fallos encontrados junto con las **capturas de pantalla reales** tomadas durante las sesiones de depuración del entorno distribuido.

1. Pérdida de Enrutamiento SSH y Reenvío de Puertos

Evidencia visual: Capturas de las reglas de VirtualBox para VM1, VM2 y VM3

Causa Raíz: Al apagar y clonar las máquinas, los puertos de escucha SSH (22) perdieron vinculación o se sobrescribieron por error con las reglas del servidor Apache (8080) o de Node (3454), generando el bloqueo `Connection Refused`.

Solución Aplicada: Se ingresó a la configuración avanzada de adaptadores de red de cada máquina virtual para crear reglas concurrentes separadas usando el botón "+".



2. Fallo de Ubicación del Script en Node.js (MODULE_NOT_FOUND)

Evidencia visual: Consola SSH con salida y detención del proceso

Causa Raíz: Invocación del intérprete fuera del directorio del backend (sobre `/home/pelado/`). Node no pudo localizar las dependencias instaladas dentro de la carpeta local `node_modules`.

Solución Aplicada: Se aseguró el posicionamiento mediante comandos de ruta antes de arrancar la API (`cd ~/api-backend && node index.js`).

```
Error: Cannot find module '/home/pelado/index.js'
  at Module._resolveFilename (node:internal/modules/cjs/loader:1134:15)
  at Module._load (node:internal/modules/cjs/loader:975:27)
  at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:128:12)
  at node:internal/main/run_main_module:28:49 {
  code: 'MODULE_NOT_FOUND',
  requireStack: []
}
```

3. Error HTTP 500 y Crasheo Crítico de Iteración (data.forEach is not a function)

Evidencia visual: Consola de Desarrollo e Interfaz de Alumnos con Error 500

Causa Raíz: Discrepancia sintáctica severa. La tabla MariaDB contenía la columna `nombre` (singular) pero el backend enviaba y buscaba el campo `nombres` (plural). Al fallar la consulta SQL, el backend capturaba el error y retornaba un objeto plano de fallo `{"error": "Error en DB"}`. El frontend colapsaba con un `TypeError` al intentar aplicar el método iterable `.forEach()` sobre un objeto en lugar de un array.

Solución Aplicada: Mapeo semántico directo en las consultas SQL de Node.js mediante el uso de alias (`nombre AS nombres`) para igualar la interfaz esperada por el cliente.

The screenshot displays a web application interface on the left and a browser's developer console on the right. The interface has two sections: 'Cargar Alumno' with input fields for 'Recalde', 'Alejandro', and '40627371', and a 'Guardar' button; and 'Lista de Alumnos' with a 'Consultar' button. The developer console shows three error messages: two 'Failed to load resource: the server responded with a status of 500 (Internal Server Error)' and one 'Uncaught (in promise) TypeError: data.forEach is not a function' at Sistema/:54:22.

4. Bloqueo de Subredes Aisladas en Modo NAT (Destination Host Unreachable)

Evidencia visual: Prueba de traza ICMP mediante Ping desde la VM2

Causa Raíz: El adaptador NAT aísla a cada máquina virtual en una red privada estricta sin puentes de comunicación directa. La VM2 (Backend) era incapaz de enviar datos a la IP `10.0.2.15` de la VM1 por ausencia de caminos de red.

Solución Aplicada: Se mapeó externamente el puerto 3306 de la VM1 hacia el host Windows físico, configurando a la VM2 para apuntar a la IP gateway `10.0.2.2`, la cual hace de puente directo.

```
pelado@Ubuntu:~/api-backend$ ping -c 4 10.0.2.15
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
From 10.0.2.16 icmp_seq=1 Destination Host Unreachable
From 10.0.2.16 icmp_seq=2 Destination Host Unreachable
From 10.0.2.16 icmp_seq=3 Destination Host Unreachable
From 10.0.2.16 icmp_seq=4 Destination Host Unreachable

--- 10.0.2.15 ping statistics ---
4 packets transmitted, 0 received, +4 errors, 100% packet loss, time 3066ms
pipe 4
```

5. Pérdida del Hilo Activo por Comandos Concurrentes e Intérprete Inadecuado

Evidencia visual: curl local de MariaDB y caída del subprocesso

Causa Raíz: Uso de combinaciones de escape (`Ctrl + C`) o llamadas de prueba directas por comandos secundarios en la misma consola SSH, lo que terminaba matando accidentalmente el subprocesso en segundo plano de Node.js.

Solución Aplicada: Se realizó la limpieza de puertos mediante `sudo killall node`, se encendió el backend de forma dedicada y se mantuvo la terminal SSH abierta en **estado de escucha fijo (congelada)** mientras se operaba el sistema desde el navegador web.

```
pelado@Ubuntu:~/api-backend$ curl http://10.0.2.2:3306
curl: (1) Received HTTP/0.9 when not allowed
pelado@Ubuntu:~/api-backend$ |
```

```
pelado@Ubuntu:~/api-backend$ nano index.js
pelado@Ubuntu:~/api-backend$ node index.js
🕒 Probando conexión a MariaDB...
🚀 Backend Node.js corriendo en el puerto 3454
✅ ¡CONEXIÓN EXITOSA! El backend ya puede hablar con la BD.
pelado@Ubuntu:~/api-backend$ curl http://10.0.2.2:3306
curl: (1) Received HTTP/0.9 when not allowed
pelado@Ubuntu:~/api-backend$ |
```

🏁 Estado de Éxito en Producción

Con las capturas analizadas e implementadas en el reporte, se constata el éxito final de la arquitectura distribuida:

- El Backend levanta de forma persistente imprimiendo: ✅ `¡CONEXIÓN EXITOSA!`
- El Frontend responde asincrónicamente desde el puerto corporativo configurado en Apache (`8080`).

- Los datos se persisten y se visualizan de manera fluida y libre de excepciones en el navegador cliente de la máquina física.